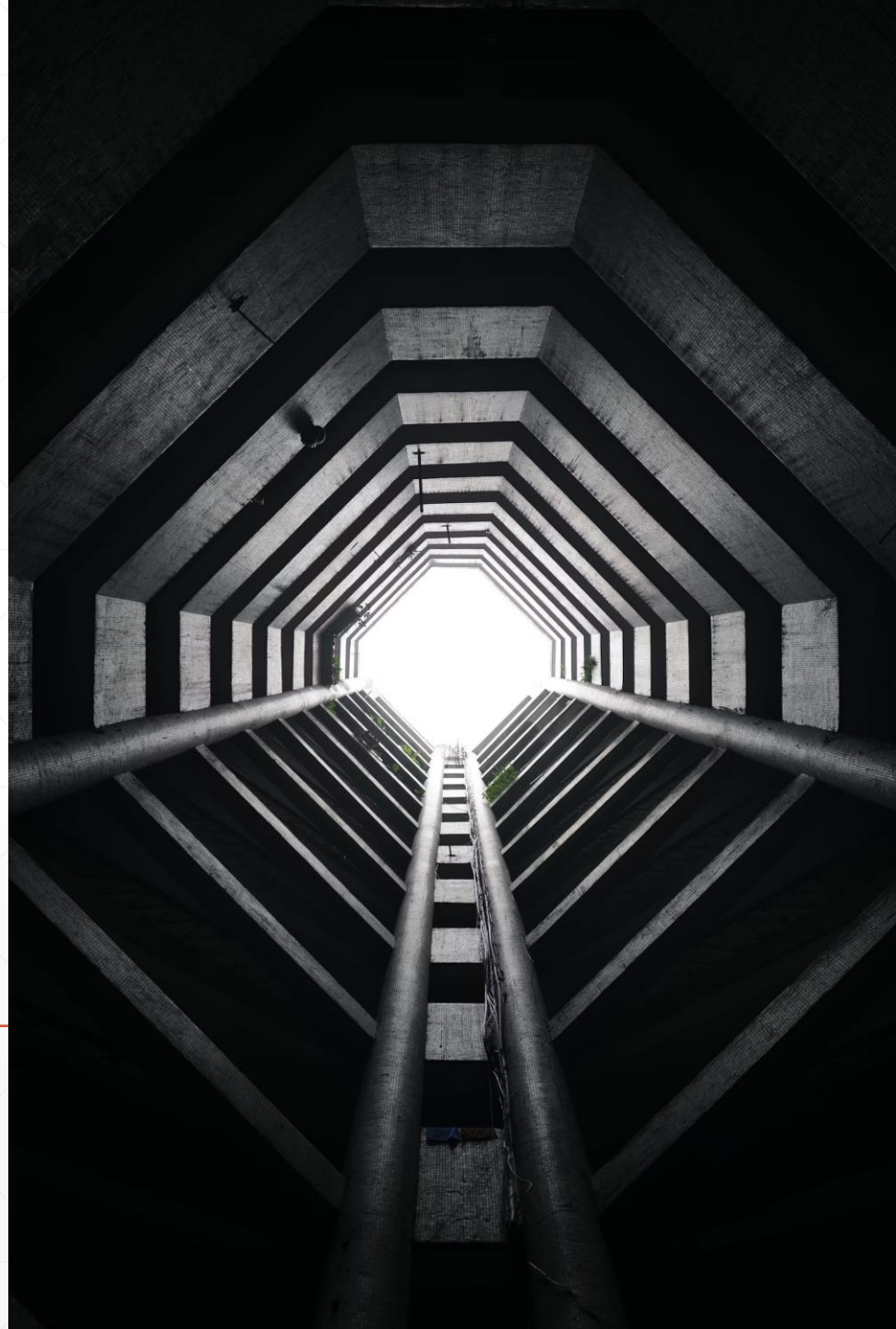


ByteBuf之四

ByteBuf进阶

北京理工大学计算机学院
金旭亮



概述



前面几讲已经把ByteBuf最基本的用法都介绍了，在本讲中，将介绍一些在某些开发场景中很有用的技术特性，同时，进一步地介绍ByteBuf相关的技术内幕。

掌握本讲所介绍的内容，有助于你真正地在实际开发中用好ByteBuf。

查看ByteBuf的内容



利用循环，可以写一个函数，用于输出ByteBuf中的内容：

```
//打印ByteBuf中的内容
public static void printByteBuf(ByteBuf byteBuf) {
    var count = byteBuf.readableBytes();
    for (int i = 0; i < count; i++) {
        System.out.print(byteBuf.getBytes(i) + ",");
    }
    System.out.println();
}
```



Netty提供了一个ByteBufUtil类，里面有一个prettyHexDump()命令，可以直观地看到ByteBuf中保存的数据（以十六进制方式表示），如下所示：

```
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+-----+
|00000000| 0b 0c 0d 0e                |....|
+-----+-----+
```

基于现有的ByteBuf创建ByteBuf视图



一旦创建好了一个ByteBuf，可以调用ByteBuf的以下方法为它创建多个“视图（View）”，这些视图，实际上访问的是**同一个**数据：

```
duplicate()  
slice()  
slice(int, int)  
readSlice(int)  
retainedDuplicate()  
retainedSlice()  
retainedSlice(int, int)  
readRetainedSlice(int)
```

每个这些方法都将返回一个新的 ByteBuf 实例，它具有自己的读写索引。其内部存储也是共享的。这使得派生缓冲区的创建成本是很低廉的，但是这也意味着，如果你修改了它的内容，也同时修改了其对应的源实例，所以要小心。



如果你了解关系型数据库中“视图”的概念，可以将其类比于此。

ByteBuffer视图示例

```
var byteBuf = Unpooled.buffer();
byte[] data = {11, 12, 13, 14};
byteBuf.writeBytes(data);
//原始数据: 11,12,13,14,
printByteBuffer(byteBuf);
System.out.println(ByteBufUtil.prettyHexDump(byteBuf));

//创建一个新视图, 不会复制原先ByteBuffer的readerIndex和writerIndex
var duplicate = byteBuf.duplicate();
//输出与前一样: 11,12,13,14,
printByteBuffer(byteBuf);
//输出新视图内容, 与前一样
System.out.println(ByteBufUtil.prettyHexDump(duplicate));

//修改第一个元素内容
byteBuf.setByte(0, (byte) 100);
//查看视图中的数据: 100,12,13,14,
printByteBuffer(duplicate);
System.out.println(ByteBufUtil.prettyHexDump(duplicate));
```

ByteBufferViewDemo.java

引用计数与资源回收



ByteBuf在不用时，需要及时回收。Netty采用了引用计数方式进行处理，当其引用计数为0时，可回收其占用的内存。由于引用计数为0的ByteBuf会被回收，因此，这时再访问它，会抛出IllegalReferenceCountException。

场景	引用计数值
创建实例	1
调用retain()方法	+1
调用release()方法	-1

必须配对使用



上述方法是实例方法，Netty还有一个ReferenceCountUtil类，里面提供了两个静态的同名方法用于维护引用计数。

引用计数示例

[1,2,3]

```
ByteBuffer buf = ByteBufferAllocator.DEFAULT.buffer();
//初始计数值: 1
System.out.println("初始计数值: " + buf.refCnt());

buf.retain();
//retain()之后, 计数值: 2
System.out.println("retain()之后, 计数值: " + buf.refCnt());

buf.release();
//release()之后, 计数值: 1
System.out.println("release()之后, 计数值: " + buf.refCnt());

//写入一个数据, 一切正常
buf.writeByte(100);

buf.release();
//再次release()之后, 计数值: 0
System.out.println("再次release()之后, 计数值: " + buf.refCnt());

//再次写入一个数据, 抛出IllegalReferenceCountException
buf.writeByte(100);
```

ByteBufferRefCount.java

ByteBuffer引用计数的大致规则如下:

- 在默认情况下, 当创建完一个ByteBuffer时, 引用计数为1; 每次调用retain()方法, 引用计数加1;
- 每次调用release()方法, 引用计数减1;
- 如果引用为0, 再次访问这个ByteBuffer对象, 将会抛出异常;
- 如果引用为0, 表示这个ByteBuffer没有哪个线程引用, 它占用的内存需要回收。

ByteBuf的回收策略



当ByteBuf的引用计数已经为0时，Netty会进行ByteBuf的回收，分为以下两种场景：

1

如果属于池化的ByteBuf内存，回收方法是：放入可以重新分配的ByteBuf池，等待下一次分配。

2

如果属于未池化的ByteBuf缓冲区，需要细分为两种情况：如果是堆（Heap）结构缓冲，会被JVM的垃圾回收机制回收；如果是直接（Direct）内存类型，则会调用本地方法（`unsafe.freeMemory`）释放外部内存。

何时需要手动释放资源?



在实际开发中，Netty提供给我们的，从通道中提取数据创建出来的ByteBuf是“池化”的ByteBuf，Netty会自动释放它们，但我们自己的创建ByteBuf，则需要考虑手动释放资源的问题。

- ▶ Netty在从Channel中读取数据时，会估计数据的大小，创建相应的ByteBuf，这个就是pipeline中第一个入站Handler的read方法参数中所接收到ByteBuf，它会在pipeline尾部的TailContext这一组件中被释放掉。
- ▶ 一种常见的作法，是在ChannelHandler的read方法中，调用其基类的channelRead()方法，将ByteBuf向后传递。如果没有调用这个方法（即中途打断了流水线），那么，你必须手动地调用release()方法释放资源。

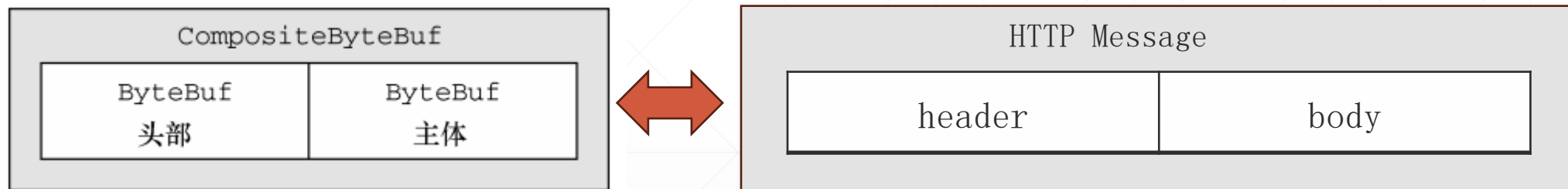


使用ByteBuf的引用计数时，要特别注意不要“多次回收”或者“忘记回收”，特别是将ByteBuf作为参数调用各种方法时，要注意这些方法内部是否修改了引用计数。

CompositeByteBuf



可以将多个ByteBuf合并为一个，构成一个“复合缓冲区（Composite ByteBuf）”，它可以避免不必要的拷贝操作。这在处理像HTTP消息这样的有着明确“分块特性”数据，非常方便。



CompositeByteBuf示例

```
//准备header部分
byte[] headerValue = ""
    content-type: text/html; charset=utf-8;
    Status Code: 200 OK;

    """.getBytes(StandardCharsets.UTF_8);
ByteBuf header = Unpooled.wrappedBuffer(headerValue);
//准备body部分
byte[] bodyValue = ""
    <html>
    <head>
        <meta charset="utf-8" />
        <title>hello</title>
    </head>
    <body>
    <h1>hello</h1>
    </body>
    """.getBytes(StandardCharsets.UTF_8);
ByteBuf body = Unpooled.wrappedBuffer(bodyValue);
```

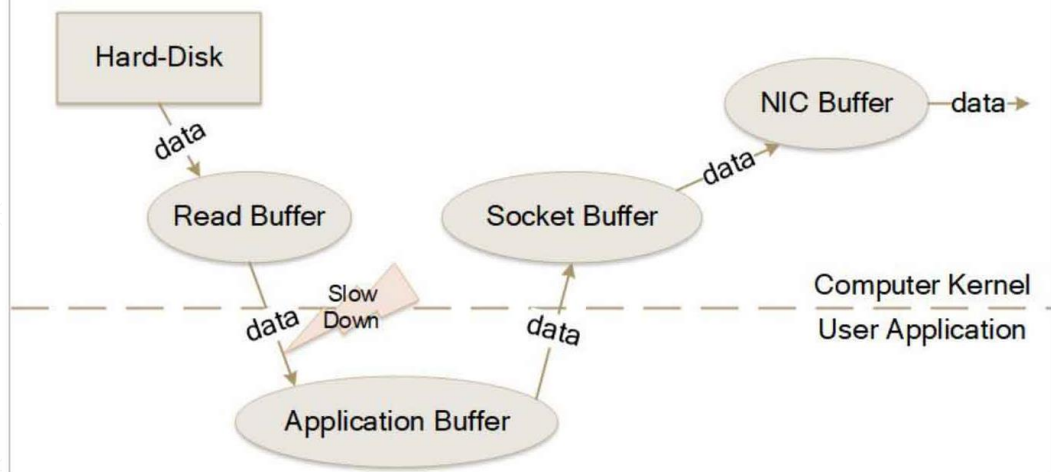
```
//构建CompositeByteBuf
CompositeByteBuf httpBuf = Unpooled.compositeBuffer();
//追加ByteBuf到CompositeByteBuf中
httpBuf.addComponent(header, body);

//读取第一个ByteBuf的内容
var readHeaderValue = httpBuf.component(cIndex: 0)
    .toString(StandardCharsets.UTF_8);
System.out.println(readHeaderValue);
//读取第二个ByteBuf的内容
var readBodyValue = httpBuf.component(cIndex: 1)
    .toString(StandardCharsets.UTF_8);
System.out.println(readBodyValue);
//减少CompositeByteBuf引用计数, 为0时被回收
httpBuf.release();
```

CompositeByteBufDemo.java

计算机网络数据传输之“零拷贝”

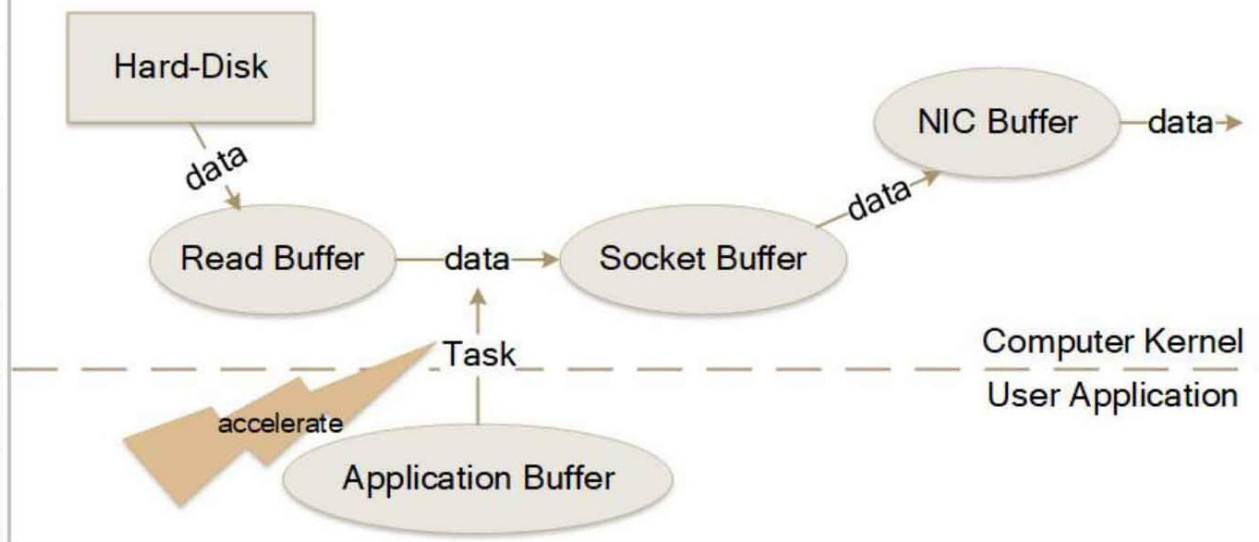
计算机网络传统方式数据传输过程



计算机在网络上发送数据时，传统方式需要将传输数据拷贝到用户空间，再复制到内核空间，这个过程由于涉及操作系统用户态与内核态的切换，是比较慢的。

“零拷贝”方式，不需要将传输数据拷贝到用户空间（User Space），而是直接在内核空间（Kernel Space）中进行传输。

计算机网络“零拷贝”数据传输过程





Netty如何实现“零拷贝”？



Netty提供CompositeByteBuf组合缓冲区类，可以将多个ByteBuf合并为一个逻辑上的ByteBuf，避免了各个ByteBuf之间的拷贝。



Netty提供了ByteBuf的浅层复制操作（slice、duplicate），可以将ByteBuf分解为多个共享同一个存储区域的ByteBuf，避免内存的拷贝。



在使用Netty进行文件传输时，可以调用FileRegion包装的transferTo()方法直接将文件缓冲区的数据发送到目标通道，避免普通的循环读取文件数据和写入通道所导致的内存拷贝问题。



在将一个byte数组转换为一个ByteBuf对象的场景下，Netty提供了一系列的包装类，避免了转换过程中的内存拷贝。



如果通道接收和发送ByteBuf都使用直接内存进行Socket读写，就不需要进行缓冲区的二次拷贝。如果使用JVM的堆内存进行Socket读写，那么JVM会先将堆内存Buffer拷贝一份到直接内存再写入Socket中，相比于使用直接内存，这种情况在发送过程中会多出一次缓冲区的内存拷贝。所以，在发送ByteBuffer到Socket时，尽量使用直接内存而不是JVM堆内存。